

浙江大学



人工智能与交互设计实验报告

项目名称：基于 HarmonyOS 的 App 开发实践

姓 名：李维扬

学 号：3230101883

学 院：信息与工程学院

专 业：电子科学与技术

指导老师：张克俊

2026 年 5 月 31 日

目 录

一	产品概述	3
1.1	背景、定位与目标用户	3
1.2	核心痛点	3
1.3	核心功能概览	3
1.4	产品特色与边界	3
二	核心功能	4
2.1	Dashboard	4
2.2	Task List	4
2.3	Add Task / Edit Task	4
2.4	Task Detail	4
2.5	本地持久化	5
2.6	优先级规则	5
三	信息架构与用户流程	6
3.1	页面结构	6
3.2	添加任务流程	6
3.3	查看任务流程	6
3.4	删除任务流程	7
3.5	完成任务流程	7
3.6	置顶与延期流程	7
四	视觉设计语言	7
4.1	设计基调	7
4.2	色彩系统	8
4.3	字体系统	9
4.4	圆角与卡片	10
4.5	阴影与层级	10
4.6	留白	10
五	自定义组件	11
5.1	TaskCard	11
5.2	PrimaryButton	11
5.3	PriorityTag	11
5.4	CategoryChip / CategoryBadge	11
5.5	CustomInput / CustomTextArea / CustomPickerField	11
5.6	EmptyState	13

5.7 IconButton / BackIconButton	14
六 交互与动画	14
6.1 按钮反馈	14
6.2 左滑删除	14
6.3 长按菜单	14
6.4 完成任务动画	15
6.5 Dashboard 推荐切换动画	15
6.6 页面转场	15
6.7 表单反馈	15
七 技术实现简述	15
7.1 HarmonyOS / ArkTS	15
7.2 本地数据存储	15
7.3 页面路由	16
7.4 规则推荐	16
7.5 AI-assisted development	16

一 产品概述

StudyFlow 是一款面向大学生的学习任务管理 App。它围绕任务记录、deadline 管理、优先级判断和完成反馈展开，目标是在本地设备上提供一个轻量、清晰、低压力的个人学习任务工作区。

1.1 背景、定位与目标用户

大学生的学习任务往往来源分散、类型混杂，而且 deadline、工作量和重要性并不总是同步显现。StudyFlow 就是一款针对于这种学习场景的个人效率工具。应用通过结构化任务信息、规则驱动优先级和清晰的移动端界面，帮助用户在多项学业任务之间建立更明确的判断依据。

主要目标用户是需要同时管理多项学业任务的大学生。典型场景包括：收到新作业后立即记录任务；在一周开始时浏览所有任务并判断先后顺序；在完成某项任务后快速更新状态。在产品策略上，StudyFlow 采用本地优先思路，不依赖登录、后端服务或实时协作功能。

1.2 核心痛点

用户群体面临三类核心问题：任务来源分散，容易遗漏；截止日期与工作量不匹配，容易低估压力；多个任务并行时优先级难以判断。StudyFlow 的目标是降低基础任务管理成本，让用户以较少操作完成添加、查看、判断和更新状态。

1.3 核心功能概览

StudyFlow 当前由 Dashboard、Task List、Add Task / Edit Task 和 Task Detail 四类页面组成。Dashboard 作为欢迎页负责每日聚焦和快速入口，Task List 负责集中浏览与管理，Add Task / Edit Task 负责结构化录入，Task Detail 负责查看单个任务的完整信息。配合本地持久化和规则优先级，这些页面构成了任务录入、判断、更新和完成的完整闭环。

1.4 产品特色与边界

StudyFlow 的特色主要体现在三个方面。第一，它面向学习场景组织任务信息，不只记录标题和完成状态，还引入类别、截止日期、预估用时、重要性和备注等字段。第二，优先级机制透明且可解释，用户可以通过 HIGH、MEDIUM、LOW 快速识别更值得关注的任务。第三，整体视觉和交互强调低压力与明确反馈，通过暖中性背景、卡片布局、手势操作和状态动画提升使用效率。

产品边界也保持明确。StudyFlow 聚焦于本地个人学习任务管理，不包含登录、云同步、多人协作、推送通知或应用商店发布等扩展功能。对本项目而言，优先保证核心功能完整、交互稳定、视觉统一和演示流畅，比引入复杂但不稳定的能力更重要。

二 核心功能

2.1 Dashboard

Dashboard 是应用首页，负责提供每日概览和快速入口。它优先告诉用户两件事：当前还剩多少任务，以及现在最值得关注的是哪一个任务。

页面包含日期与问候语、Remaining / Completed 统计卡、Suggested Focus 推荐卡片，以及进入 Add Task 和 Task List 的入口。用户还可以直接在 Suggested Focus 上切换完成状态，或长按执行 Delay 1 Day，因此首页同时承担少量高价值的直接操作。

2.2 Task List

Task List 是应用的主要工作区页面，负责集中浏览、筛选和管理当前任务。

页面顶部包含搜索框、状态筛选按钮和类别 chips。任务以自定义 TaskCard 呈现，卡片展示完成按钮、标题、描述、优先级、类别、deadline 标签和置顶图标。列表按 deadline 由近到远排序，无 deadline 的任务排在后面，置顶任务则在当前筛选结果中优先显示。

这一页也是手势和动画最集中的区域。用户可以点击圆形按钮切换完成状态，可以左滑删除，也可以长按打开置顶 / 取消置顶 / 删除菜单，因此 Task List 是主要的日常操作页面。

2.3 Add Task / Edit Task

Add Task 用于创建任务，Edit 模式用于修改编辑，复用同一套表单结构。两者可以视为同一套输入系统：一个负责录入，一个负责修正。

表单包含标题、描述、类别、截止日期、截止时间、预计时长和重要性等字段。标题是必填项，按钮会在标题为空时保持禁用。单行输入、多行输入和日期时间字段都已统一样式，以保证 Add 与 Edit 的输入体验一致，并减少格式错误。

2.4 Task Detail

Task Detail 用于展示单个任务的完整信息，是列表页与编辑页之间的稳定落点。

页面展示任务标题、描述、优先级、类别、截止时间、预计时长、状态、重要性和创建时间等信息。顶部提供返回和编辑入口，底部提供切换完成状态和删除任务的主要

操作。用户可以从 Dashboard 或 Task List 进入 Detail，再进入 Edit 模式，编辑完成后返回 Detail 查看更新结果。

2.5 本地持久化

StudyFlow 采用本地优先的数据策略。任务数据保存在 HarmonyOS 的 Preferences 中，因此应用不依赖网络、账户或后端服务也能完成完整任务管理。TaskStore 负责统一处理读写、序列化和旧数据兼容，核心逻辑如下：

```
1  //...
2  const TASKS_JSON_KEY = 'tasks_json';
3
4  async saveTasks(): Promise<void> {
5    const tasksJson = JSON.stringify(this.tasks);
6    this.preferencesStore.putSync(TASKS_JSON_KEY, tasksJson);
7    this.preferencesStore.flush((err: BusinessError) => {
8      //...
9    });
10 }
11
12 async addTask(input: AddTaskInput, context: common.UIAbilityContext):
13 Promise<StudyTask> {
14   //...
15   this.tasks.push(task);
16   await this.saveTasks();
17   return task;
18 }
```

本地持久化是每次核心任务操作之后立即执行的标准流程。

2.6 优先级规则

在测试阶段，StudyFlow 使用规则驱动自动优先级机制。系统会结合 deadline、预计时长和重要性，计算出 HIGH、MEDIUM、LOW 三档结果，并在任务卡、详情页和 Dashboard 推荐中统一使用。核心实现位于 DeadlineRules.ets：

```
1  //...
2  export function calculateTaskPriority(
3    deadline: string,
4    estimatedHours: number,
5    importance: number,
6    nowMs: number = Date.now()
7  ): string {
8    const baseLevel = getBasePriorityLevel(estimatedHours, importance);
9    const boostedLevel = Math.min(2, baseLevel +
10   getDeadlineBoostLevel(getHoursUntilDeadline(deadline, nowMs)));
11    return getPriorityLabel(boostedLevel);
12 }
```

这套规则透明、可解释，也便于在新增、编辑、排序和推荐逻辑中复用，预留了进一步优化的空间。同时，项目将“优先级”和“时间紧迫度”概念分开处理：前者用于表达整体重要程度，后者更多通过 deadline 文案和颜色系统体现。

三 信息架构与用户流程

3.1 页面结构

- Dashboard：应用首页，无返回栏。
- Task List：工作区页面，使用轻量 Header。
- Add Task / Edit Task：二级操作页面，使用标准 TopBar。
- Task Detail：二级详情页面，使用标准 TopBar。

StudyFlow 采用四页核心结构：Dashboard、Task List、Add Task / Edit Task、Task Detail。它们按“首页概览 - 工作区 - 二级操作 - 单任务详情”的层次组织。Dashboard 负责每日概览和分流，Task List 承担主要浏览与管理，Add / Edit 负责输入，Detail 负责完整查看与任务级操作。

3.2 添加任务流程

- (1) 用户从 Dashboard 右上角加号，或从 Task List 底部 Add New Task 按钮进入 Add Task 页面。
- (2) 用户填写标题、描述、类别、截止日期、截止时间、预计时长和重要性。
- (3) 表单根据标题是否为空控制 Create Task 按钮可用状态，避免创建无效任务。
- (4) 用户点击创建后，任务被写入本地存储，同时计算优先级。
- (5) 页面返回 Task List，新任务立即出现在当前列表中。

这里没有在保存后回到 Dashboard 而是直接进入 Task List，因为新增任务后的下一步通常是确认任务已经进入列表，或继续整理其它任务。

3.3 查看任务流程

查看任务流程对应的是先看摘要再看完整信息的浏览顺序。

- (1) 用户可以从 Dashboard 的 Suggested Focus 卡片，或从 Task List 的任意任务卡片单击进入 Task Detail。
- (2) Detail 页面展开显示任务标题、描述、类别、优先级、截止时间、预计时长、重要性、状态和创建时间。
- (3) 用户可以在确认任务信息后返回来源页面，也可以点击编辑入口进入 Edit 模式。

3.4 删除任务流程

- (1) 用户可以在 Task List 中左滑任务卡片直接删除。
- (2) 也可以长按任务卡片打开弹窗后选择删除。
- (3) 如果用户已经进入 Task Detail，也可以在详情页底部直接删除当前任务。
- (4) 删除后列表会通过退场、收起和上浮动画实时刷新，减少突兀感。

3.5 完成任务流程

完成任务流程强调状态确认和动画反馈。

- (1) 用户点击任务卡片左上角圆形按钮，或在 Detail 页面底部切换任务状态。
 - (2) 在 Task List 中，卡片会先显示完成确认状态，再执行柔和退场、下沉或重排动画。
 - (3) 在 Dashboard 中，Suggested Focus 卡片完成后，当前卡片退场，下一个推荐任务再延迟浮现。
 - (4) 状态变更被写入本地存储，Task List、Task Detail 和 Dashboard 统计同步刷新。
- 这一流程保留了更明确的视觉确认，更有交互感和完成感，也使状态切换更可靠。

3.6 置顶与延期流程

置顶与延期属于辅助操作，用来补充系统默认排序和推荐逻辑。

- (1) 在 Task List 中，用户长按任务卡片可以打开操作菜单，执行置顶、取消置顶或删除。
- (2) 被置顶的任务会在当前筛选结果中优先显示，并在标题前显示图钉图标。
- (3) 在 Dashboard 中，用户长按 Suggested Focus 卡片，可以打开弹窗将已有 deadline 的任务延迟一天。
- (4) 延期后，任务 deadline、紧迫度颜色和推荐结果会实时刷新。

置顶代表用户的强制排序意图，延期则代表用户对现实时间安排的主动修正，比系统自动规则优先级更高。二者结合后，系统规则与用户主观控制保持了相对平衡。

四 视觉设计语言

4.1 设计基调

StudyFlow 的视觉方向可以概括为 calm productivity。界面整体保持安静、克制、偏 daily focus，不做强商业化包装，也不使用复杂数据面板。具体上主要依赖三个原则：

色彩克制，只在 deadline 紧迫时提升强度；层级清晰，通过暖中性背景、近白卡片、衬线标题和无衬线正文建立区分；反馈柔和，让动画和状态色明显但不过度刺激。

4.2 色彩系统

- Primary color: #D27760，用于主要按钮、完成状态和关键交互反馈。
- Background color: #F5F4F0，用于全局页面背景。
- Secondary / neutral colors: #8E8A83、#E8E5DF 等暖灰色，用于次级文本、边框、空状态和弱化信息。
- 背景色：暖中性米色，用于承载页面整体基调。
- 卡片色：近白色和轻微暖白渐变，用于任务卡、统计卡和输入控件。
- 主操作色：陶土色，用于主要按钮、完成状态和关键交互反馈。
- 状态色：根据 deadline 紧急程度区分 overdue、24h 内、1-3 天、4-10 天和普通任务。
- 弱化色：暖灰色，用于说明文本、空状态和已完成任务。

StudyFlow 的基础色板保存在 DesignTokens.ets 中。整体色彩以暖中性为底，陶土色为主操作色。基础界面色如下：

```
1 export const SF_COLOR_BACKGROUND: string = '#F5F4F0';
2 export const SF_COLOR_CARD: string = '#FFFFFF';
3 export const SF_COLOR_PRIMARY_TEXT: string = '#33312E';
4 export const SF_COLOR_SECONDARY_TEXT: string = '#8E8A83';
5 export const SF_COLOR_MUTED_TEXT: string = '#4D4A46';
6 export const SF_COLOR_BORDER: string = '#E8E5DF';
7 export const SF_COLOR_PRIMARY_ACTION: string = '#D27760';
8 export const SF_COLOR_PRIMARY_FOREGROUND: string = '#FFFFFF';
```

其中，#F5F4F0 用于全局背景，#FFFFFF 用于卡片和输入表面，#33312E 与 #8E8A83 分别承担主文本和次级文本，#D27760 作为主操作色，边框统一使用 #E8E5DF。

除基础色之外，TaskCard 还定义了一套时间紧迫度颜色系统。普通任务、即将到期任务和逾期任务会获得不同的卡面渐变、边框和阴影：

```
1 // standard
2 export const SF_COLOR_TASK_CARD_SURFACE_START: string = '#FAFAF9';
3 export const SF_COLOR_TASK_CARD_SURFACE_MID: string = '#FFFFFF';
4 export const SF_COLOR_TASK_CARD_SURFACE_END: string = '#FFFFFF';
5
6 // overdue
7 export const SF_COLOR_TASK_CARD_OVERDUE_SURFACE_START: string =
8   '#FFE4E6';
9 export const SF_COLOR_TASK_CARD_OVERDUE_SURFACE_MID: string =
10   '#FFF1F2';
```

```
9 export const SF_COLOR_TASK_CARD_OVERDUE_SURFACE_END: string =  
  '#FFFFFF';  
10 export const SF_COLOR_TASK_CARD_OVERDUE_BORDER: string = '#FFC9D1';  
11  
12 // critical / soon / upcoming  
13 export const SF_COLOR_TASK_CARD_CRITICAL_BORDER: string = '#FFCCD3';  
14 export const SF_COLOR_TASK_CARD_SOON_BORDER: string = '#FFD7B8';  
15 export const SF_COLOR_TASK_CARD_UPCOMING_BORDER: string = '#F0D8C2';
```

deadline 胶囊也有独立色系：中性 deadline 使用灰米渐变，1-3 天内转为橙色系，逾期转为粉红系：

```
1 export const SF_COLOR_TASK_CARD_DEADLINE_NEUTRAL_START: string =  
  '#E7E5E4';  
2 export const SF_COLOR_TASK_CARD_DEADLINE_NEUTRAL_END: string =  
  '#F5F5F4';  
3 export const SF_COLOR_TASK_CARD_DEADLINE_UPCOMING_START: string =  
  '#FF637E';  
4 export const SF_COLOR_TASK_CARD_DEADLINE_UPCOMING_END: string =  
  '#FF8904';  
5 export const SF_COLOR_TASK_CARD_DEADLINE_SOON_START: string = '#FFB86B';  
6 export const SF_COLOR_TASK_CARD_DEADLINE_SOON_END: string = '#FF8904';  
7 export const SF_COLOR_TASK_CARD_DEADLINE_OVERDUE_START: string =  
  '#FF7A8E';  
8 export const SF_COLOR_TASK_CARD_DEADLINE_OVERDUE_END: string =  
  '#FFA1AD';
```

优先级和紧迫度分开表达，避免所有语义都堆在同一个颜色通道里。

4.3 字体系统

- Newsreader：用于 Dashboard 问候语、页面标题、部分首页区块标题和辅助斜体文本。
- Inter：用于正文、按钮、输入、标签、任务卡片内容和系统信息。
- 数字默认使用比例宽度，避免普通行文中的数字间距过散。

StudyFlow 使用两套字体：Newsreader 用于展示型标题，Inter 用于正文和 UI 控件。实际字体 token 如下：

```
1 export const SF_FONT_FAMILY_DISPLAY: string =  
  'StudyFlowNewsreaderDisplay';  
2 export const SF_FONT_FAMILY_DISPLAY_STRONG: string =  
  'StudyFlowNewsreaderDisplayStrong';  
3 export const SF_FONT_FAMILY_DISPLAY_ITALIC: string =  
  'StudyFlowNewsreaderItalic';  
4 export const SF_FONT_FAMILY_BODY: string = 'StudyFlowInter';  
5  
6 export const SF_FONT_PAGE_TITLE: number = 24;
```

```
7 export const SF_FONT_SECTION_TITLE: number = 18;
8 export const SF_FONT_BODY: number = 16;
9 export const SF_FONT_META: number = 14;
10 export const SF_FONT_META_SMALL: number = 12;
11 export const SF_FONT_TAG: number = 12;
12 export const SF_FONT_TAG_SYSTEM: number = 11;
```

Newsreader 主要用于 Dashboard 问候语、页面级标题、Dashboard 区块标题和少量斜体辅助文本；Inter 则承担正文、输入、按钮、标签、元信息和任务内容。

数字默认使用比例宽度，不启用全局 tabular figures，以避免普通日期、时长和计数文本出现不自然的数字空隙。

4.4 圆角与卡片

界面大量使用 16 左右的大圆角卡片，任务属性 badge 使用更小的圆角矩形，以区别于可交互筛选 chips。卡片和主要输入控件统一使用 16 的基础圆角，胶囊类元素使用极大半径形成完整 pill：

```
1 export const SF_RADIUS_BASE: number = 16;
2 export const SF_RADIUS_PILL: number = 999;
3 export const SF_BUTTON_HEIGHT: number = 48;
4 export const SF_INPUT_HEIGHT: number = 50;
```

这样可以让主要容器保持稳定、柔和的矩形轮廓，同时让筛选 chips、deadline 胶囊和状态标签具备更明确的胶囊感。

4.5 阴影与层级

阴影整体保持轻量，用于区分卡片和浮层。常规卡片阴影非常克制，逾期卡片和浮层会适度增强：

```
1 export const SF_COLOR_TASK_CARD_SHADOW: string = '#0D6E6A63';
2 export const SF_COLOR_TASK_CARD_OVERDUE_SHADOW: string = '#22FFCCD3';
3 export const SF_COLOR_TASK_CARD_OVERDUE_SHADOW_PULSE: string =
  '#55FF9AAA';
4 export const SF_COLOR_TASK_CARD_CRITICAL_SHADOW: string = '#2BFFE4E6';
5 export const SF_COLOR_TASK_CARD_SOON_SHADOW: string = '#24FFD7B8';
6 export const SF_COLOR_TASK_CARD_UPCOMING_SHADOW: string = '#18D27760';
```

在组件实现中，任务卡通常使用小半径、低透明度阴影；弹窗则使用更明确的白底和竖向偏移，让其从内容层中被识别出来。

4.6 留白

页面级横向留白统一使用 24，卡片内部基础留白使用 16：

```
1 export const SF_SPACE_PAGE: number = 24;  
2 export const SF_SPACE_CARD: number = 16;
```

在实际应用中, Dashboard 的留白更强调呼吸感和首屏节奏; Task List 的留白则更偏功能导向, 重点是让筛选区、任务卡和底部按钮之间保持清晰, 但又不牺牲列表密度。

五 自定义组件

5.1 TaskCard

TaskCard 是 Task List 中最核心的信息组件。它负责承载完成按钮、标题、描述、优先级、类别、deadline 标签和置顶图标, 并支持点击进入详情、左滑删除、长按菜单和完成动画。卡片结构可以把任务状态、时间压力和操作入口放在同一个稳定视觉单元中。

5.2 PrimaryButton

PrimaryButton 用于页面主操作, 例如 Create Task、Save Changes、Add New Task、Mark as Complete 等。它统一主色、圆角、按压反馈和禁用状态, 保证不同页面的关键操作具有一致的视觉权重和交互反馈。

5.3 PriorityTag

PriorityTag 用于展示 High / Medium / Low 优先级。它采用小矩形 badge 样式, 全局统一为大写文案, 并在 TaskCard 中使用任务卡专属颜色变体。

优先级标签同时和可点击的筛选 chips 区分开。

5.4 CategoryChip / CategoryBadge

CategoryChip 用于筛选和选择, 是可交互控件。CategoryBadge 用于任务属性展示, 是静态信息标签, 两者在形状、颜色和语义上区分开。刻意拆成两类组件: Chip 表示可选择的交互控件, Badge 表示任务自身属性。这样可以避免用户把静态标签误认为筛选按钮。

5.5 CustomInput / CustomTextArea / CustomPickerField

三个组件共同承担表单输入系统。

其中, CustomInput 负责单行文本录入, CustomTextArea 负责多行描述, CustomPickerField 负责日期和时间这类只读可点击字段。三者共享统一的边框、圆角、字号和禁用态规则, 从而保证 Add Task 与 Edit 模式的输入体验一致。

CustomInput 的核心是把输入文字、leading icon、边框颜色和禁用状态统一:

```

1  //...
2  @Component
3  export struct CustomInput {
4    @Prop value: string = '';
5    @Prop placeholder: string = '';
6    @Prop isEnabled: boolean = true;
7    @Prop hasError: boolean = false;
8    @Prop leadingIcon: string = '';
9
10   build() {
11     Row({ space: 10 }) {
12       //...
13       TextInput({
14         placeholder: this.placeholder,
15         text: this.value
16       })
17       .fontSize(SF_FONT_BODY)
18       .fontWeight(FontWeight.Regular)
19       .fontFamily(SF_FONT_FAMILY_BODY)
20       .placeholderColor(CUSTOM_INPUT_PLACEHOLDER_COLOR)
21       .backgroundColor(SF_COLOR_CARD)
22       .border({ width: 0 })
23       .onChange((value: string) => {
24         if (this.isEnabled) {
25           this.onChange(value);
26         }
27       })
28     }
29     .height(SF_INPUT_HEIGHT)
30     .backgroundColor(SF_COLOR_CARD)
31     .borderRadius(SF_RADIUS_BASE)
32     .border({
33       width: 1,
34       color: this.getBorderColor()
35     })
36   }
37 }

```

CustomTextArea 单独控制高度和顶部对齐，避免文本垂直居中带来的阅读问题：

```

1  //...
2  const CUSTOM_TEXTAREA_HEIGHT: number = 122;
3
4  @Component
5  export struct CustomTextArea {
6    build() {
7      TextArea({
8        placeholder: this.placeholder,
9        text: this.value
10      })
11      .height(CUSTOM_TEXTAREA_HEIGHT)
12      .fontSize(SF_FONT_BODY)
13      .fontFamily(SF_FONT_FAMILY_BODY)
14      .borderRadius(SF_RADIUS_BASE)
15      .border({
16        width: 1,
17        color: this.getBorderColor()
18      })

```

```

19     .padding({
20       left: 12,
21       right: 12,
22       top: 8,
23       bottom: 12
24     })
25     .align(Alignment.TopStart)
26   }
27 }

```

日期和时间字段不允许自由文本输入，因此项目将它们实现为统一的 picker 壳体：

```

1  //...
2  @Component
3  export struct CustomPickerField {
4    @Prop text: string = '';
5    @Prop isEnabled: boolean = true;
6    @Prop leadingIcon: string = '';
7
8    build() {
9      Row({ space: 10 }) {
10        //...
11        Text(this.text)
12          .layoutWeight(1)
13          .fontSize(SF_FONT_BODY)
14          .fontFamily(SF_FONT_FAMILY_BODY)
15      }
16      .height(SF_INPUT_HEIGHT)
17      .backgroundColor(SF_COLOR_CARD)
18      .borderRadius(SF_RADIUS_BASE)
19      .border({
20        width: 1,
21        color: SF_COLOR_BORDER
22      })
23      .onClick(() => {
24        if (this.isEnabled) {
25          this.onTap();
26        }
27      })
28    }
29  }

```

5.6 EmptyState

EmptyState 用于无任务、搜索无结果、分类无结果和状态筛选无结果等场景。它直接显示在页面背景上，不使用白色卡片，以降低提示层级；同时通过 compact 模式兼容局部空状态提示。

```

1  //...
2  @Component
3  export struct EmptyState {
4    @Prop title: string = '';
5    @Prop message: string = '';
6    @Prop compact: boolean = false;

```

```
7
8  build() {
9      Column({ space: this.compact ? 6 : 8 }) {
10         Text(this.title)
11             .fontSize(this.getTitleFontSize())
12             .fontWeight(FontWeight.Regular)
13             .fontFamily(SF_FONT_FAMILY_BODY)
14             .fontColor(SF_COLOR_SECONDARY_TEXT)
15             .textAlign(TextAlign.Center)
16
17         Text(this.message)
18             .fontSize(this.getMessageFontSize())
19             .fontFamily(SF_FONT_FAMILY_BODY)
20             .fontColor(SF_COLOR_SECONDARY_TEXT)
21             .textAlign(TextAlign.Center)
22             .lineHeight(this.compact ? 18 : 20)
23     }
24     .padding({
25         top: this.getVerticalPadding(),
26         bottom: this.getVerticalPadding(),
27         left: this.getContainerPadding(),
28         right: this.getContainerPadding()
29     })
30 }
31 }
```

5.7 IconButton / BackIconButton

IconButton 统一图标按钮的尺寸、热区、边框、选中态和按压反馈。BackIconButton 用于页面级返回，保证返回图标在不同页面中一致。单独封装后，返回、编辑、筛选和添加等入口可以共享同一套点击热区和按下效果，不依赖文本符号或页面局部样式。

六 交互与动画

6.1 按钮反馈

按钮和图标按钮在按下时会通过颜色变深、局部缩放或轻微阴影变化提供即时反馈，帮助用户确认点击已经生效。

6.2 左滑删除

Task List 中的任务卡片支持左滑删除。达到阈值后，卡片会淡出、下沉并收起高度，下方任务顺次上浮补位。

6.3 长按菜单

Task List 中长按任务卡片会弹出轻量菜单，提供置顶、取消置顶和删除；Dashboard 中长按 Suggested Focus 则提供 Delay 1 Day。菜单采用白底浮层样式，不打断当前页面流程。

6.4 完成任务动画

完成任务动画分为短暂停顿和退场补位两个阶段。用户点击完成按钮后，圆形按钮会缩放回弹并显示勾号，标题同步弱化；随后卡片退场或列表重排，让完成状态先被确认，再进入下一步动画。

6.5 Dashboard 推荐切换动画

Dashboard 的 Suggested Focus 使用更柔和的替换动画。当前推荐任务完成后，旧卡片先下沉淡出，再由新推荐任务缓慢浮现，表达关注项已更新。

6.6 页面转场

页面转场主要依赖系统默认的左右滑动动画。实现上尽量使用真实路由栈返回，而不是替换页面实例，以保证前进与返回方向一致。这样既减少了额外动画实现成本，也让整体导航体验更稳定。

6.7 表单反馈

表单反馈主要通过按钮禁用、placeholder 弱化和字段样式统一来表达。标题为空时，Create Task 或 Save Changes 保持不可用；保存后则通过页面返回和数据刷新让用户立即看到结果，不额外引入复杂提示框。

七 技术实现简述

7.1 HarmonyOS / ArkTS

项目基于 HarmonyOS ArkTS 实现，界面由 ArkUI 组件构建，整体在 DevEco Studio 中开发。应用主体由 DashboardPage、TaskListPage、AddTaskPage 和 TaskDetailPage 等.ets 文件组成，并配合统一的 DesignTokens 与自定义组件实现页面复用。

7.2 本地数据存储

任务数据通过 Preferences 存储在本地。TaskStore 负责加载、保存、增删改查、完成状态切换、置顶和延期等核心数据操作，从而保证应用在无网络条件下仍可独立运行。

7.3 页面路由

应用使用 `router` 完成页面跳转。Dashboard 作为首页, TaskList 作为工作区, Add / Edit 和 Detail 作为二级页面; 返回操作优先使用 `router.back()`, 以保持系统默认转场一致。

7.4 规则推荐

Suggested Focus 在测试阶段不接入 AI 模型, 而是基于本地任务数据和规则排序生成。规则主要考虑任务状态、置顶状态、deadline 远近、priority 和 importance, 用于从当前任务中选出最值得关注的一项。

7.5 AI-assisted development

开发过程中使用 AI (Codex) 辅助进行代码修改、问题定位, 但 AI 不参与应用运行时功能; 应用本身不包含云端 AI、登录或同步能力。